

Homework 4 - Double Pendulum

Βέμης Νικόλαος 16 January 2026

1 Trajectory Generation

The trajectory generation script works exactly as it did in the 3rd assignment, but the mass, length and torque limit parameters have been updated to the following values:

$$\text{mass} = [0.1055, 0.0762] \text{ kg}, \quad \text{length} = [0.05, 0.05] \text{ m. Torque limits} = [-0.12, 0.12] \text{ Nm.}$$

Additionally, the $\mathbf{Q}$ and $\mathbf{R}$ matrices used in the optimal trajectory generation have been modified to:

$$\mathbf{Q} = \text{diag}([100, 100, 1, 1]), \quad \mathbf{R} = \text{diag}([50, 000, 50, 000]).$$

We can notice that the $\mathbf{R}$ matrix has been scaled up significantly. We did this because the torque limits allow the system to apply enough torque to overcome gravity and simply do a half rotation without needing to build up momentum (can be seen in 2), which is not the point of the exercise. So by increasing the $\mathbf{R}$ matrix values, we penalize high torques more, which encourages the optimizer to find trajectories that build up momentum instead of relying on high torques. This also allows us to keep the relatively high torque limits that will help us when dealing with disturbances during tracking.

Another change is that the time horizon has been *decreased* to $T = 2.0$ s (from $T = 5.0$ s in the previous assignment). This is because with the new parameters, the system prefers to do a half swing motion and/or hold the half upright position for a long time, resulting in a long period of the actuators being strained in order to hold the weigh hanging downwards or just doing a half swings. This can be seen in the 1 figure below.

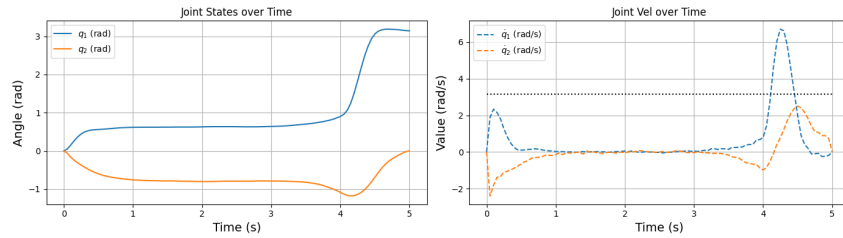


Figure 1: Trajectory with long time horizon $T = 5.0$ s. The system swings up and then holds the position, straining the actuators.

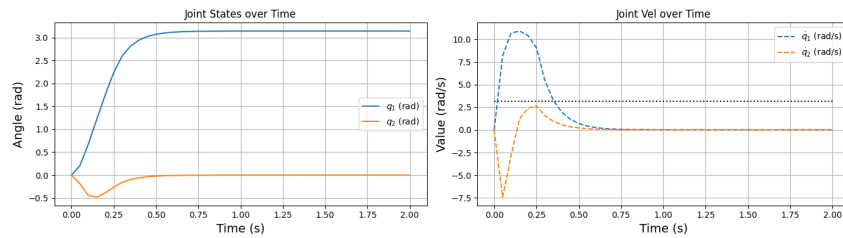


Figure 2: Trajectory with small $\mathbf{R}$ matrix values. The system shoots up quickly using high torques instead of building momentum.

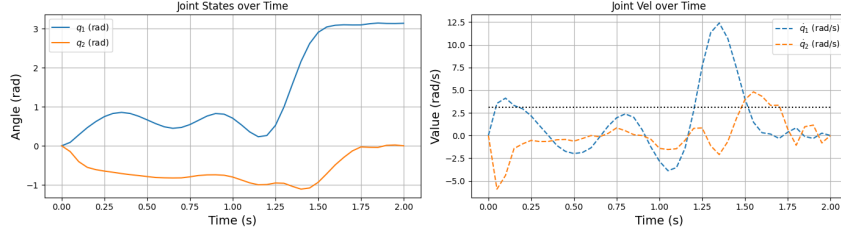


Figure 3: Trajectory with reduced time horizon $T = 2.0$ s and the aforementioned $\mathbf{R}$ matrix. The system builds momentum and swings up quickly.

By reducing the time horizon, we force the optimizer to dismiss such solutions and converge straight to the goal. The final trajectory used for tracking can be seen in 3.

2 Real-time Torque Controller

The script `part4.py` implements the simulation of the real-time torque controller for the double pendulum system and uses the same controller structure as the one in the `control.py` script used in the real system.

This script implements a real-time torque controller for two motors over a CAN bus using the `pyCandle` interface and it relies on offline-computed data:

- A gain schedule $K[i]$ (stored in `K_matrix.npy`)
- A reference state trajectory $x_{\text{ref}}[i]$ (stored in `trajectory.csv`)
- A reference input (feedforward torque) trajectory $u_{\text{ref}}[i]$ (stored in `inputs.csv`)

with $i = 0, \dots, N - 1$ indexing time steps along the trajectory. The dimensions of these arrays are:

$$x_{\text{ref}}[i] \in \mathbb{R}^4, \quad u_{\text{ref}}[i] \in \mathbb{R}^2, \quad K[i] \in \mathbb{R}^{2 \times 4}, \quad i = 0, \dots, N - 1.$$

The control architecture is designed as a hybrid discrete-time controller operating at a frequency of $f_{\text{ctrl}} = 20$ Hz, while the physics simulation advances at a finer resolution Δt_{sim} (100 Hz or 1000 Hz). This necessitates a sub-stepping approach where the physics engine performs $N = \Delta t_{\text{ctrl}} / \Delta t_{\text{sim}}$ steps for every single control update (here $N = 5$).

The controller operates on a finite state machine logic, switching dynamically between **Nominal Tracking** and **Recovery** based on the deviation from the reference trajectory.

3 Control Modes

The system state $\mathcal{S}$ is determined by the Euclidean distance d between the current state $\mathbf{x}_{\text{curr}}$ and the target reference state $\mathbf{x}_{\text{ref}}$. Let δ_{th} be the deviation threshold.

- **Nominal Tracking Mode:** Active when the system is within the safety corridor ($d < \delta_{th}$) and the trajectory has not ended. The controller sequentially follows the pre-calculated trajectory and increments the trajectory index k at each control step. All while using the pre-computed gain matrix $\mathbf{K}_k$ associated with the specific time-step.
- **Recovery Mode:** Triggered when the system deviates significantly ($d \geq \delta_{th}$) or drifts while holding the top position. In this mode, the controller abandons the sequential tracking and searches for the nearest valid state in the reference trajectory to re-engage. It then uses k -Nearest Neighbor interpolation to compute a stabilizing gain matrix $\mathbf{K}_{\text{interp}}$.

4 Control Law

The control input $\mathbf{u}$ is calculated using a feedback law with feed-forward compensation and hard actuator saturation.

Given the current error $\mathbf{e} = \mathbf{x}_{curr} - \mathbf{x}_{des}$, the raw control input is:

$$\mathbf{u}_{raw} = \mathbf{u}_{ff} - \mathbf{K} \cdot \psi(\mathbf{e}) \quad (1)$$

Where:

- $\mathbf{u}_{ff}$ is the feed-forward torque from the reference trajectory.
- $\mathbf{K}$ is the gain matrix (either static or interpolated).
- $\psi(\cdot)$ is the topological wrap-around function ensuring angular errors remain within $[-\pi, \pi]$:

$$\psi(\theta) = (\theta + \pi \pmod{2\pi}) - \pi \quad (2)$$

Finally, the output is clipped to hardware limits u_{max} :

$$\mathbf{u}_{total} = \text{clip}(\mathbf{u}_{raw}, -u_{max}, +u_{max}) \quad (3)$$

5 Averaging method for gain interpolation

When the controller enters **Recovery Mode** simply picking the single nearest gain matrix can lead to discontinuity in control action. We did not use the Mahalanobis distance-based interpolation method described in the previous assignment because it could not respond well to large deviations from the trajectory. Instead, we employ a k -Nearest Neighbors (k -NN) inverse-distance weighted interpolation.

First, to handle the periodicity of the joint angles correctly during distance calculation, the state vector $\mathbf{x} = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top$ is mapped to a feature space embedding $\Phi(\mathbf{x})$ involving sine and cosine components:

$$\Phi(\mathbf{x}) = [\sin \theta_1, \cos \theta_1, \sin \theta_2, \cos \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top$$

The distance d_j between the current state $\mathbf{x}$ and the j -th point in the reference trajectory is computed as the Euclidean distance in this feature space:

$$d_j = \|\Phi(\mathbf{x}) - \Phi(\mathbf{x}_{ref}[j])\|_2$$

The algorithm selects the set $\mathcal{I}$ of the k indices with the smallest distances (typically $k = 5$). The interpolated gain matrix $\mathbf{K}_{interp}$ is computed as a weighted average:

$$\mathbf{K}_{interp} = \sum_{j \in \mathcal{I}} \bar{w}_j \mathbf{K}[j]$$

where the normalized weights $\bar{w}_j$ are derived from the inverse distance:

$$w_j = \frac{1}{d_j + \epsilon}, \quad \bar{w}_j = \frac{w_j}{\sum_{m \in \mathcal{I}} w_m}$$

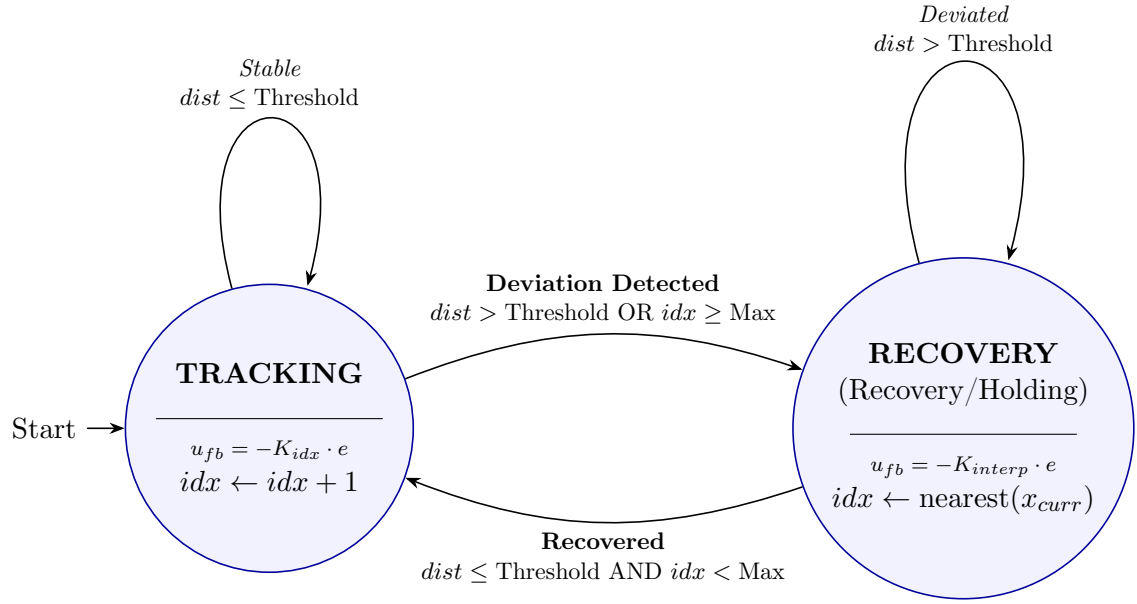
Here, ϵ is a small constant (10^{-6}) to prevent division by zero. This method ensures that the controller applies feedback gains that are most relevant to the current dynamic state of the system, smoothing the re-entry into the trajectory.

6 Algorithm Definition

The following pseudocode details the logic implementation equivalent to the deployed Python controller.

Algorithm 1: Hybrid Tracking and Recovery Control Loop

Input: $\mathbf{x}_{\text{ref}}$ (Ref. State), $\mathbf{u}_{\text{ref}}$ (Feed-forward), $\mathbf{K}$ (Gains), $\mathbf{x}_0$ (Initial State)
Data: δ_{th} (Deviation Threshold), u_{max} (Torque Limit)
 $idx \leftarrow 0$
 $\mathbf{x}_{curr} \leftarrow \mathbf{x}_0$
 $mode \leftarrow \text{TRACKING}$
while *Simulation Running* **do**
 // Check Tracking Status
 $d \leftarrow \text{dist}(\mathbf{x}_{curr}, \mathbf{x}_{\text{ref}}[idx])$
 $holding \leftarrow (idx \geq idx_{max})$
 if $d > \delta_{th}$ **or** $holding$ **then**
 $mode \leftarrow \text{RECOVERY}$
 $idx \leftarrow \text{FindNearestIndex}(\mathbf{x}_{curr}, \mathbf{x}_{\text{ref}})$ // Reset index
 end
 else
 $mode \leftarrow \text{TRACKING}$
 end
 // Select Gains and Targets
 $\mathbf{x}_{des} \leftarrow \mathbf{x}_{\text{ref}}[idx]$
 $\mathbf{u}_{ff} \leftarrow \mathbf{u}_{\text{ref}}[idx]$
 if $mode == \text{TRACKING}$ **then**
 $\mathbf{K}_{gain} \leftarrow \mathbf{K}[idx]$
 $idx \leftarrow idx + 1$
 end
 else
 $\mathbf{K}_{gain} \leftarrow \text{KNN_Interpolate}(\mathbf{x}_{curr}, \mathbf{x}_{ref}, \mathbf{K}, k = 5)$
 end
 // Compute Control Law
 $\mathbf{e} \leftarrow \text{WrapToPi}(\mathbf{x}_{curr} - \mathbf{x}_{des})$
 $\mathbf{u}_{fb} \leftarrow -\mathbf{K}_{gain} \cdot \mathbf{e}$
 $\mathbf{u}_{total} \leftarrow \text{Clip}(\mathbf{u}_{ff} + \mathbf{u}_{fb}, -u_{max}, u_{max})$
 if $t_{stop_start} \leq t \leq t_{stop_end}$ **then**
 $\mathbf{u}_{total} \leftarrow [0, 0]$
 end
 // Physics Sub-stepping
 for $s \leftarrow 0$ **to** sim_steps **do**
 $\mathbf{x}_{curr} \leftarrow \text{RK4}(\mathbf{x}_{curr}, \mathbf{u}_{total}, \Delta t_{sim})$
 end
 $\mathbf{x}_{curr} \leftarrow \text{AddNoise}(\mathbf{x}_{curr}) + \text{WrapAngles}(\mathbf{x}_{curr})$
end



7 Results

The results can be seen in the `part4.ipynb` notebook and in the video file.

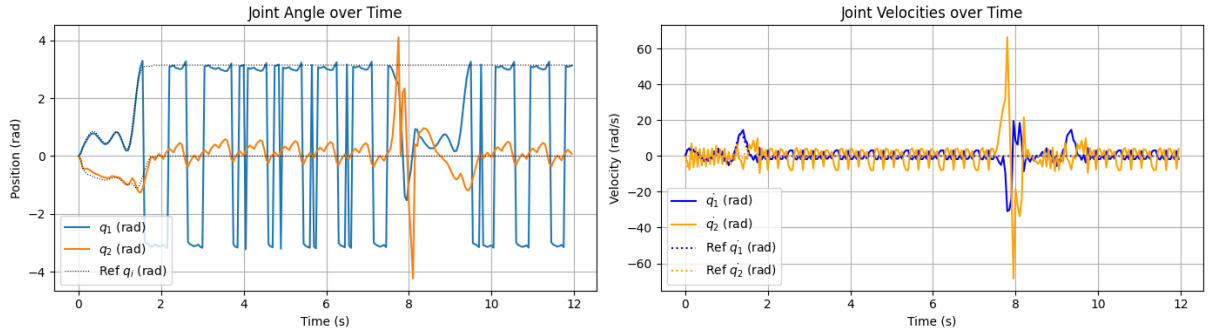


Figure 4: Recovery from a large disturbance at $t = 7.5$ s to $t = 7.9$ s. The controller quickly brings the system back to the trajectory.